

RAVEN

Verified Context Passports for the Agentic Web

Technical overview

TL;DR

RAVEN is a recipient-aware, decision-preserving context compressor that sits in FRONT of LLM agents. It compresses at two edges - user-memory -> agent (Passport) and agent -> agent (RELAY) - cutting context tokens ~80-93% while GUARANTEEING that standing constraints survive. It is deterministic, stdlib-first, torch-free, and measured with the model's REAL input-token usage. Live as a uAgent on Fetch.ai Agentverse, as a web A/B demo, and as an MCP server inside Claude.

The problem: context logistics, not model IQ

Two failure modes of today's agent stacks:

1. Broadcast cost. The common pattern dumps the WHOLE user memory into EVERY agent of a multi-agent system. With N agents that is $N \times$ the full payload, every turn, every user.
2. Lost-in-the-middle. A capable model handed 1,783 tokens of notes recommended an Italian restaurant for a vegetarian - the dietary rule was buried and dropped. More context is not safer context. RAVEN reframes the bottleneck: decide WHO needs to know WHAT. You don't brief a chef and an accountant with the same memo.

Pipeline (deterministic, no LLM in the compressor)

```
raw memory / notes
| ingest: split into sentences
v
atomic typed facts      (multi-label: 1 sentence -> N typed facts)
|                      types: dietary, permission, budget_limit,
|                      expense_receipt, availability, location,
|                      preference, other
v
BM25 retrieval          (query-aware: ranked by the role's task)
v
recipient-aware select  (keep facts whose type the role needs)
v
critical-fact GUARD     (force-keep standing-rule types, even score 0)
v
dedup -> render passport -> count tokens (real model usage)
```

Every fact carries a TYPE and a reason it was kept (guard vs. relevant), so the output is fully explainable - no black-box compressor.

Recipient model

Each role declares the fact TYPES it needs. A fact enters a passport only if its type matters to that role AND it is query-relevant - except guarded types, which are always kept.

role	needs these fact types	GUARD (force-kept)
-----	-----	-----
restaurant	dietary, preference, budget_limit, loc	dietary
calendar	availability	availability
budget	budget_limit, permission	budget_limit, permission
writer	dietary, budget_limit, availability, permission, preference, location	dietary, budget_limit, availability, permission

Why the GUARD is the moat

A constraint can be lexically IRRELEVANT to the query ('vegetarian' vs. 'where to eat') yet decision-critical. Pure relevance-based compression drops it - exactly the lost-in-the-middle failure. The guard force-keeps standing-rule TYPES even at BM25 score 0, so the loss only ever lands on low-relevance facts, never on a rule.

Edge 1 - Passport (user memory -> agent)

Same memory, different role => different passport. Worked example (memory ~151 tokens):

```
INPUT: "Maya is vegetarian and allergic to peanuts. Keep dinners under
       $40. Free Fri after 7pm, in lab until 5:30. Always confirm before
       paying. Concert tickets $65. I prefer Thai and Italian food."
```

```
role: budget      -> 46 tok (-69%)  HARD: under $40 ; RISK: confirm before pay
                                   (drops vegetarian, schedule, $65 receipt)
role: calendar    -> 45 tok (-70%)  only the schedule (free after 7 / lab 5:30)
role: restaurant -> 65 tok (-57%)  dietary + preferences + location
```

Least-privilege by construction: the calendar agent never even sees the budget or the allergy.

Edge 2 - RELAY (agent -> agent handoff)

On a handoff RAVEN forwards the LATEST upstream message VERBATIM (a message-floor that role-filtering can never drop) PLUS a compressed recipient-aware passport of the prior back-context.

```
handoff =  render_passport(prior_context, to_role)      # compressed
           + 'LATEST FROM UPSTREAM:\n' + latest_message # verbatim floor
```

```
full transcript 8,560 / last-msg only 183 / RELAY 889 = ~90% smaller
standing constraints survive 3/3 hops   (last-message-only: 1/3)
```

Measurement methodology (honest by construction)

The headline number is the model's OWN usage.input_tokens (read from the Anthropic API), not a self-estimate. Calls are disk-cached so demos are instant and free.

```
saved% = (1 - T_raven / T_full) * 100      # T = real input tokens
```

Baselines - a defensible comparison

raw_once	full memory serialized once (optimistic floor)
raw_broadcast	full memory to EVERY agent (real multi-agent cost)
generic_truncation	naive role-unaware truncation (weak baseline)
generic_factstore_unaware	RAVEN's fact store, role-UNAWARE, at RAVEN's exact token budget (the FAIR baseline)

The honest headline (we say this out loud)

At an EQUAL token budget, RAVEN ties the generic baseline on raw token count - by design. The moat is NOT the token number; it is DECISION QUALITY and constraint-safety at that budget. We do not claim 'RAVEN is 90% smaller than other compressors' - that would be false. The 80-93% is vs. broadcasting full memory.

Results (reproducible from a clean checkout)

M1 payload gate	raw_broadcast 11,032 -> RAVEN 786 = 92.9% reduction (vs raw_once 2,758: RAVEN total 71.5% lower)
A/B live tokens	1,783 -> 317 = ~82% fewer
A/B sweep (x5)	82-90% fewer; every standing rule kept; adversarial "charge my card now" -> REFUSED
M2 decision (equal budget)	raw 5/5 @ 8,308 tok generic 4/5 @ 960 tok (drops 'confirm before paying') RAVEN 5/5 @ 1,022 tok
RELAY	full 8,560 / last 183 / RELAY 889 = ~90% smaller
corpus 38 items / 127 facts ; Claude Haiku 4.5, temp 0, disk-cached	

Research grounding (lightweight, explainable variants)

We implemented	Grounded in
-----	-----
query-aware extractive selection	LongLLMLingua (we use BM25, NOT a learned token classifier)
inter-agent comms pruning (RELAY)	AgentPrune / "Cut the Crap"
guideline learning, no fine-tune	ACON (verifier learns 'keep type X')
constraint vs. accuracy split	CDCT (deterministic gold scoring)
near-duplicate anchoring (dedup)	SeCo

What we deliberately did NOT do (and why)

- No LLMLingua-2 learned token classifier -> we stay torch-free, fast, and EXPLAINABLE.
- No Cache-to-Cache KV compression -> listed as future work.

The extractive + deterministic choice is exactly WHY we can PROVE decision-preservation: a black-box compressor cannot show that it kept the \$40 cap and the confirm-before-pay rule.

The optional verifier (ACON-style)

A defense-in-depth checker probes raw-context vs. passport answers and learns per-role 'always keep type X' guidelines. It only fires when a critical type is GENUINELY missing - in the benchmark the static guard already covers everything, so it adds 0 tokens. Its value is shown by a dedicated test where it repairs a deliberately dropped fact.

Fetch.ai integration

RAVEN runs as a uAgent (uagents + uagents-core) speaking the standard Chat Protocol (ChatMessage / ChatAcknowledgement / TextContent / EndSessionContent). It is mailbox-connected to Agentverse, registered on the Almanac, and discoverable via ASI:One. A local Bureau demo shows two agents handing off via RELAY.

The agent makes ZERO LLM calls

The Agentverse agent is pure deterministic compression - so it needs NO API key to run, is instant, and cannot leak a key. The host model (in ASI:One / Claude) is the LLM; RAVEN just returns the compressed passport. This is the right separation for a shared public agent.

Robust chat parsing (a real bug we fixed + regression-tested)

ASI:One prepends '@agent1q...' and sends a single line. The parser strips the mention, matches role/task/memory markers ANYWHERE (':' or '='), validates the role, and treats plain text as memory. The earlier bug that defaulted everything to 'writer' (and leaked the mention) is fixed, with tests for the exact single-line/@mention case.

Three deploy surfaces, one engine

1. Agentverse uAgent (Chat Protocol) - the agentic-web surface.
2. Web A/B demo (FastAPI + Next.js): same prompt with vs. without RAVEN, REAL input-token counts, an animated atomize -> rank -> guard -> drop -> passport pipeline, and a live/cached badge.
3. MCP server: compress_memory / relay_handoff / list_roles, usable inside Claude Desktop / Claude Code / Cursor - keyless, no LLM calls.

Engineering rigor

- Stdlib-first Python 3.13; a custom BM25 retriever (zero ML downloads).
- 113 offline tests (2 skipped), deterministic.
- Strict offline-isolation invariant: the test suite imports NO uagents / fastapi / markdown / mcp / network. The heavy transports (api.py, raven_agent.py, mcp/server.py) are the SOLE importers of their SDKs; the pure logic is tested separately.
- Disk-cached LLM calls -> demos are instant and free; a cooldown prevents quota burn.
- Every kept fact carries a reason (guard vs. relevant) - fully auditable output.

Honest limitations

- Compression is lossy by definition; we bound the loss to low-relevance facts and never to standing rules (the guard).
- BM25 has vocabulary-mismatch failures ('Italian place' vs. 'where to eat'); mitigated by the recipient model + guard. Learned relevance is an optional future upgrade behind the same explainable interface.
- ASI: One auto-discovery is best-effort (the host model routes by description).
- Savings scale with input size; tiny inputs may not net-compress, and the tool reports that honestly.

What's next

- LLM API proxy (one-line base_url swap) so any app gets the savings with no rewrite.
- Learned relevance (embeddings / small classifier) behind the same interface.
- KV-level relay (Cache-to-Cache) for repeated back-context.
- LangChain / CrewAI adapters so multi-agent builders get RELAY for free.

One line

Don't broadcast the whole memory. Give each agent a VERIFIED passport - the minimal slice it needs, with the rules guaranteed. ~80-93% fewer tokens, decisions preserved.